

**LAPORAN PRAKTIKUM
TEKNOLOGI IoT 2**
“Tugas 1”



DISUSUN OLEH
Nama : Naurah Miftah Frizkiyah
Kelas : TRI 3B
NPM: 25782049

DOSEN :
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

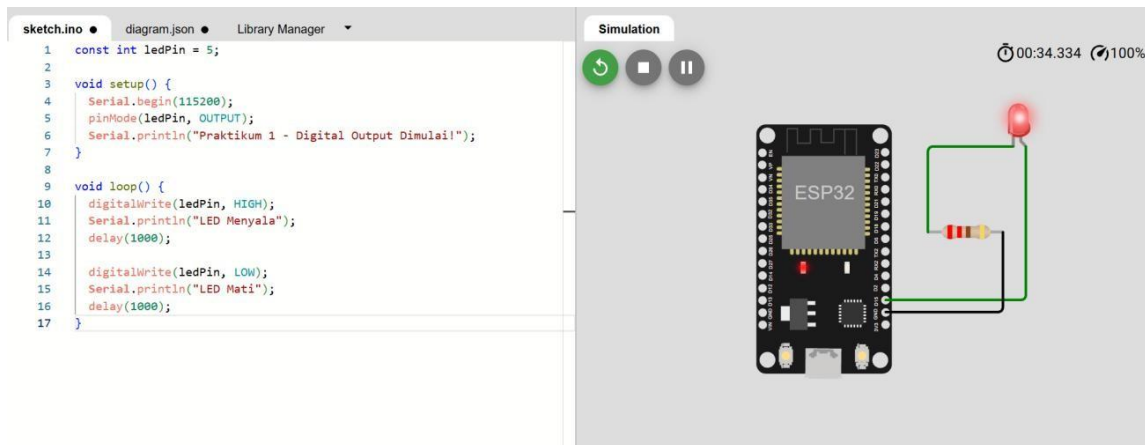
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026

➤ HASIL PELAKSANAAN PRAKTIKUM

Praktikum 1: Digital Output (Menyalakan LED)

Tujuan: Mengontrol keluaran digital pada pin mikrokontroler untuk menyalakan dan mematikan LED secara berkala.

Berikut hasil implementasi simulasi di wokwi:

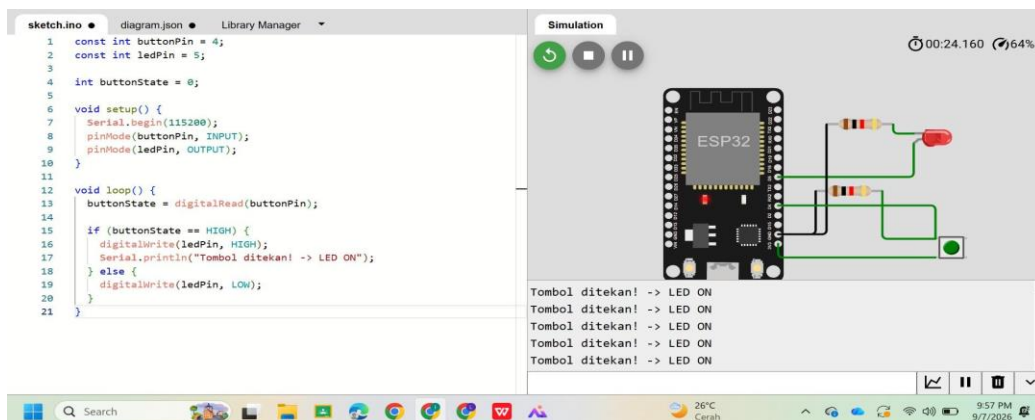


Gambar di atas menunjukkan hasil simulasi rangkaian Praktikum 1 yang berjalan. Terlihat LED merah menyala karena pin GPIO 5 mengirimkan sinyal logika HIGH (3.3V). Aliran arus listrik mengalir dari pin D5 melewati resistor 220 ohm sebagai pembatas arus, menuju kaki anoda LED, dan kembali ke GND. Karena perintah delay (1000) dipasang setelah kondisi HIGH dan LOW, LED berkedip bergantian secara teratur setiap 1 detik.

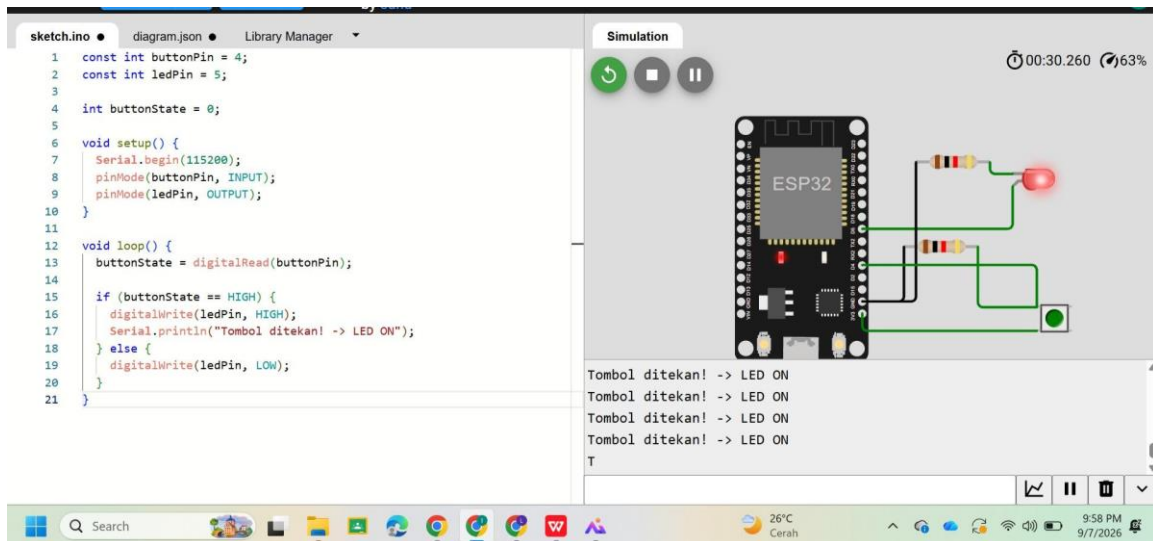
Praktikum 2: Digital Input (Membaca Push Button)

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor pull-down eksternal.

Berikut ini adalah tampilan sebelum push button ditekan, LED belum menyala:



Berikut adalah tampilan saat push button ditekan maka LED akan menyala seperti pada gambar:



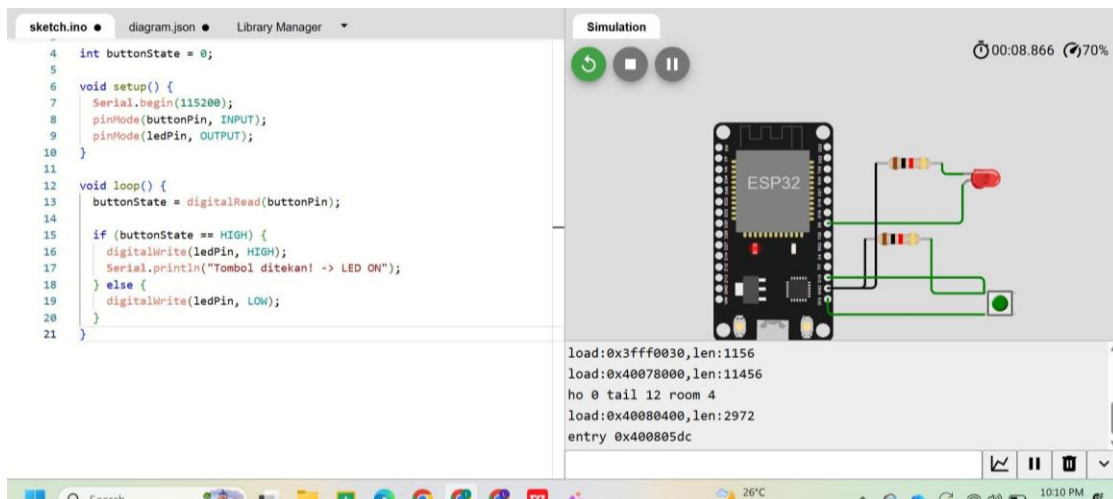
Pada pengujian ini, sistemnya menggunakan sistem momentary (aktif hanya saat pas ditekan). Jadi, pas tombol hijau diklik dan ditahan, pin D4 bakal langsung mendeteksi sinyal HIGH atau 1. Nah, ESP32 bakal langsung merespon dengan ngirim tegangan ke pin D5, makanya LED merah langsung nyala. Di saat yang bersamaan, Serial Monitor juga akan menampilkan tulisan "Tombol ditekan! -> LED ON" sebagai bukti kalau pembacaan input digitalnya berhasil.

➤ HASIL JAWABAN LATIHAN (LATIHAN 1 - 4)

Latihan 1: Pemahaman Logika Pin

Soal Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / pull-down), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut tepat pada detik di mana NodeMCU baru diberi daya?

Berikut ini adalah hasil pengujian dari latihan 1:



Jawaban: NodeMCU/ESP8266 akan mengalami gagal booting atau macet jika tombol Active-High ditekan saat perangkat pertama kali dinyalakan. Hal ini terjadi karena pin D8 (GPIO 15) bertindak sebagai strapping pin yang wajib berada dalam kondisi LOW (0V) saat power-up. Menekan tombol tersebut akan memaksa pin bernilai HIGH (3.3V), sehingga mengacaukan syarat utama untuk masuk ke mode booting normal.

Latihan 2 (Implementasi Kode):

Soal : Modifikasi kode pada Praktikum 1 agar LED berkedip secara tidak simetris, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah.

Berikut adalah hasil dari latihan 2 dengan mengganti kode program dari praktikum 1 dengan

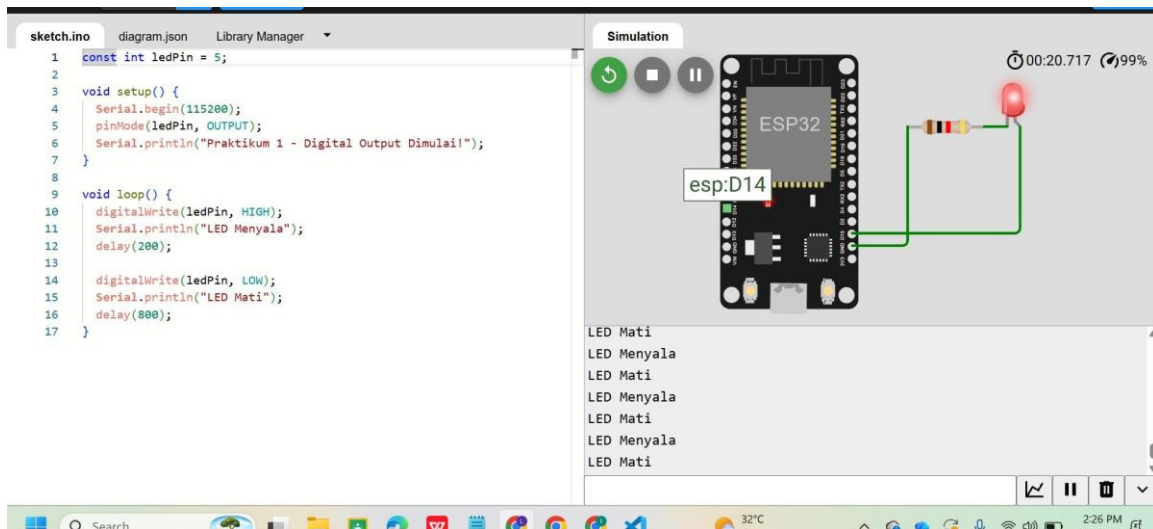
kode program:

```
void loop() { digitalWrite(ledPin, HIGH);
Serial.println("LED Menyala");
delay(200);
digitalWrite(ledPin, LOW);
```

```
Serial.println("LED Mati");
```

```
delay(800);
```

```
}
```

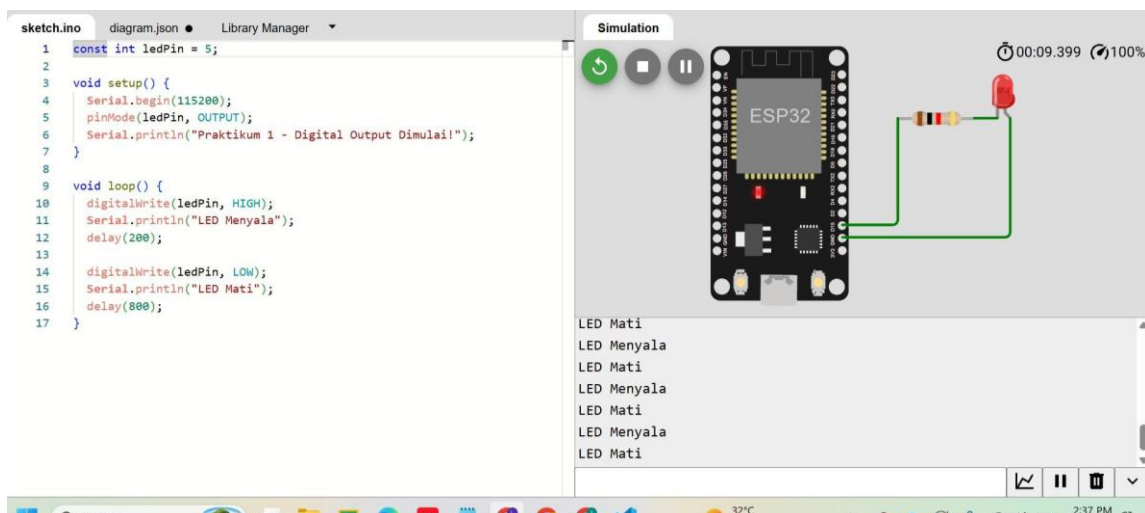


Berbeda dengan Praktikum 1 yang berkedip (1 detik ON / 1 detik OFF), pada latihan ini durasi jeda diubah menggunakan delay(200) saat LED menyala dan delay(800) saat LED mati. Hasilnya, LED terlihat berkedip dengan cepat dan langsung mati dalam waktu yang lebih lama secara berulang-ulang di dalam fungsi loop()

Latihan 3 (Analisis Sirkuit):

Soal : Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami error kompilasi? Mengapa LED tidak menyala?

Berikut adalah hasil simulasi yang telah saya lakukan:



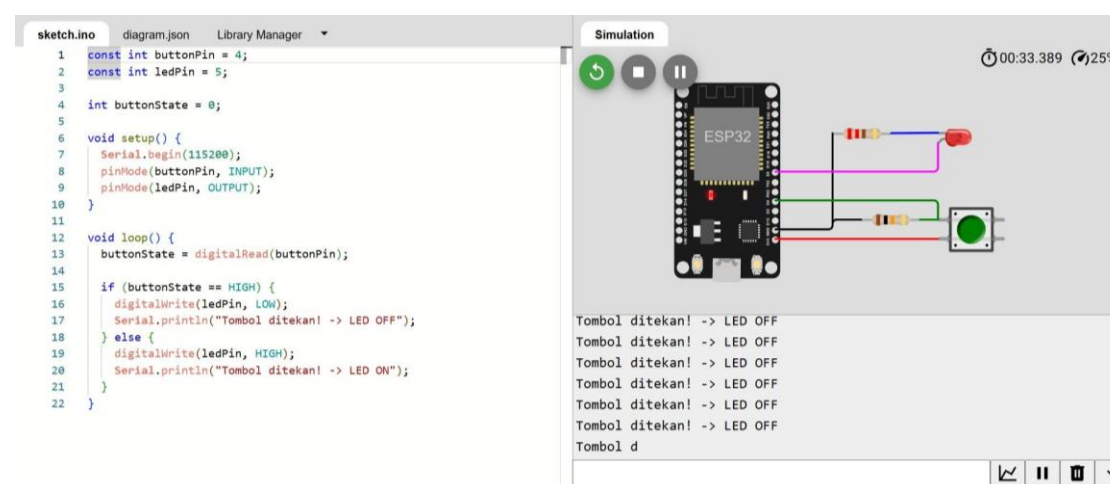
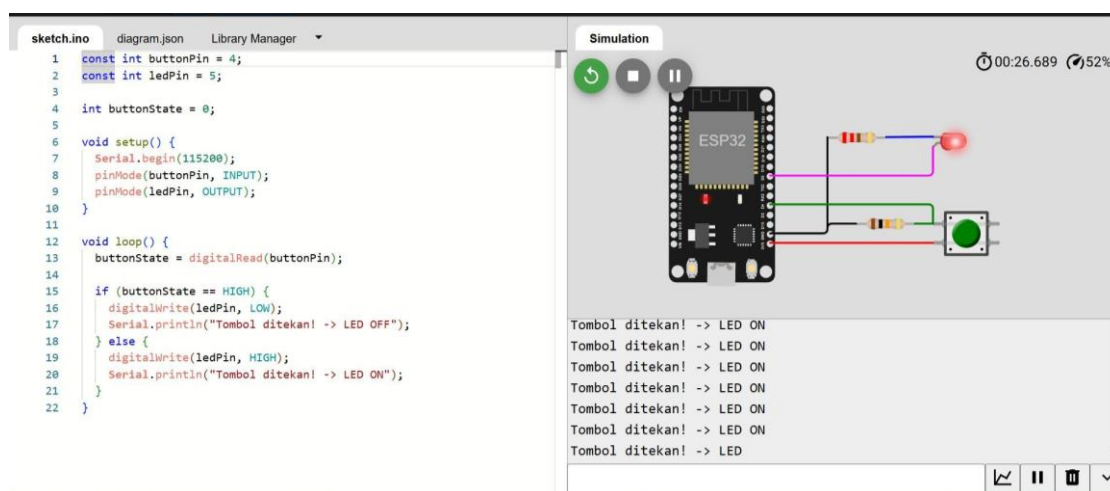
Pada simulasi ini, program tetap berjalan normal (Compile Success) karena tidak terdapat kesalahan sintaksis pada kode. Namun, LED tidak memancarkan cahaya akibat pemasangan kaki yang terbalik. Sifat dasar LED sebagai komponen semikonduktor memerlukan bias maju.

(Forward Bias) agar arus dapat mengalir. Karena terpasang dalam kondisi bias mundur (Reverse Bias), aliran arus listrik terhenti sehingga LED tidak menyala.

Latihan 4 (Pengembangan Algoritma):

Soal : Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."

Berikut adalah hasil pengembangan algoritma nya ini adalah posisi saat push button belum di tekan dan sudah ditekan pada :

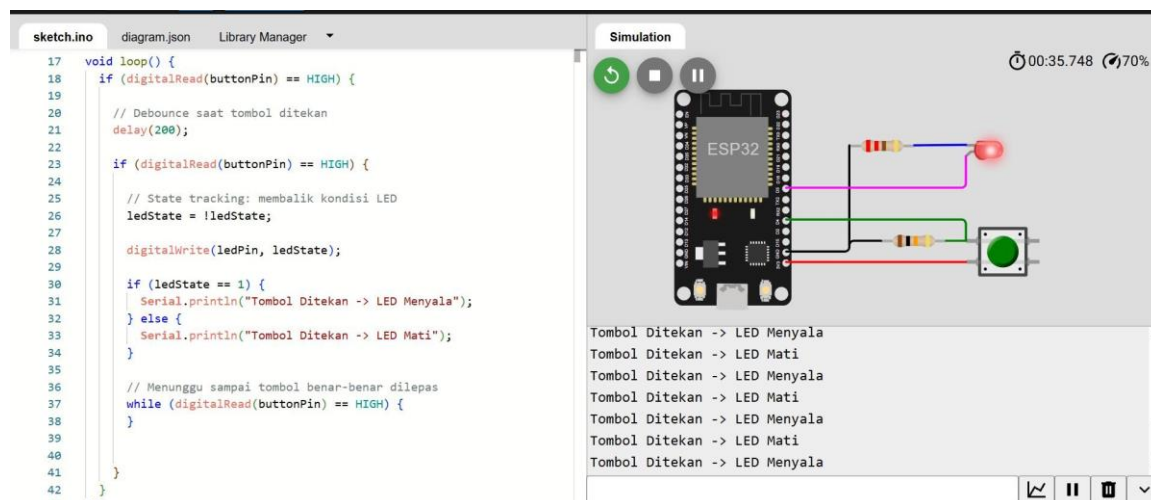
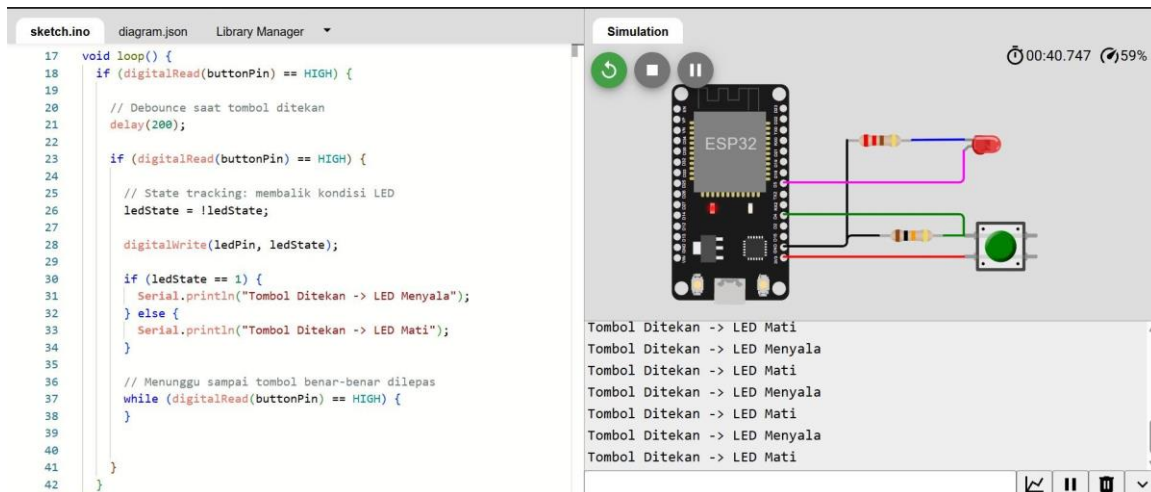


Kedua gambar di atas menunjukkan hasil yang menerapkan logika terbalik. Berbeda dengan kondisi standar, saat tombol tidak ditekan ($\text{buttonState} == 0$), ESP32 memberikan luaran logika 1 sehingga LED langsung menyala. Ketika tombol ditekan ($\text{buttonState} == 1$), program akan menjalankan perintah `digitalWrite(ledPin, 0)` yang mematikan LED.

➤ Tugas: Modifikasi Sistem Sakelar Toggle (Latching)

Tujuan: Mengimplementasikan logika pelacakan status (state tracking) pada ESP32 untuk membedakan perilaku tombol momentary dengan latching (sakelar toggle).

Berikut adalah tampilan saat push button/sakelar di tekan dia akan menyala dan kalau di tekan sekali lagi akan mati:



Gambar di atas menunjukkan pengujian sistem sakelar toggle (latching) pada simulator Wokwi. Saat tombol ditekan satu kali lalu dilepas, mikrokontroler mendeteksi adanya transisi sinyal masukan dari 0 ke 1 (rising edge). Program kemudian mengubah variabel penjejak status `ledState` menjadi 1 untuk menyalakan LED dan menampilkan pesan konfirmasi pada Serial Monitor. LED akan tetap menyala secara stabil hingga tombol mendapat instruksi tekanan berikutnya untuk memadamkan lampu. Ketika ditekan kembali satu kali lagi, LED padam dan tetap padam secara stabil tanpa mengalami flickering.

➤ **Kesimpulan**

Dari seluruh percobaan yang sudah dilakukan pada praktikum ini, dapat disimpulkan bahwa pengontrolan komponen digital pada mikrokontroler sangat bergantung pada kombinasi logika kode dan pemahaman sirkuit fisik. Pada bagian output digital, pengujian membuktikan bahwa LED bisa diatur mati dan menyalnya secara presisi menggunakan sinyal biner 1 dan 0, di mana durasi kedipannya bisa dengan mudah kita manipulasi lewat pengaturan jeda delay. Sementara pada bagian input digital, penggunaan resistor pull-down pada push button terbukti penting untuk memastikan sinyal yang dibaca mikrokontroler selalu stabil dan tidak terganggu oleh sinyal liar.

Selain itu, praktikum ini juga memberikan pemahaman penting mengenai karakteristik komponen dan pemilihan pin hardware. Kesalahan fisik seperti memasang kaki LED terbalik atau menekan tombol pada pin khusus seperti GPIO 15 saat booting tidak membuat program error saat di-compile, melainkan menyebabkan kegagalan fungsi fisik seperti LED mati atau sistem yang menggantung. Terakhir, pengembangan sistem sakelar toggle berhasil direalisasikan dengan menerapkan logika state tracking untuk mengingat kondisi lampu, deteksi transisi rising edge agar tombol tidak merespons berlebihan saat ditahan, serta pemberian jeda debounce untuk menstabilkan pantulan sinyal mekanis pada tombol.